

Search
Home
GitHub
CLASSES
ErrorBoundary
EVENTS
SIGINT
unhandledRejection
NAMESPACES
apiConfig
contactoEmergenciaService
diarioService
GLOBAL
404_Error_Handler
ACTIVIDADES
API_URL
ASPECTOS_COGNITIVOS
ActionCard
ActivitySelector
App
AuthButtons
AuthLayout
Button
CORS_Configuration
Card
Carousel
Checkbox
CircularProgress
CognitionSelector
Diario
DiaryBanner
DiaryEditor
DiaryEntry
EMOCIONES
EmergencyButton
EmergencyModal
EmotionSelector
EmotionStats
Footer
GET /

frontend/src/pages/Diario.jsx

```
1.  /**
2.   * @file Página del Diario.
3.   * @description Permite a los usuarios crear, ver, editar, eliminar
4.   * @requires react
5.   * @requires ../hooks
6.   * @requires ../service/diario.service
7.   * @requires ../components/molecules/DiaryEditor
8.   * @requires ../components/molecules/DiaryEntry
9.   * @requires ../components/atoms/Button
10.  * @requires ../components/atoms/Input
11.  * @requires ../styles/Diario.css
12.  */
13.  import React, { useState, useEffect } from "react";
14.  import { useToast } from "../hooks";
15.  import { diarioService } from "../service/diario.service";
16.  import DiaryEditor from "../components/molecules/DiaryEditor";
17.  import DiaryEntry from "../components/molecules/DiaryEntry";
18.  import Button from "../components/atoms/Button";
19.  import Input from "../components/atoms/Input";
20.  import "../styles/Diario.css";
21.
22.  /**
23.   * @function Diario
24.   * @description Componente principal de la página del diario. Gestio
25.   * @returns {JSX.Element} La página del diario.
26.   */
27.  const Diario = () => {
28.    const { success: showSuccess, error: showError } = useToast();
29.    const [entries, setEntries] = useState([]);
30.    const [isLoading, setIsLoading] = useState(false);
31.    const [showEditor, setShowEditor] = useState(false);
32.    const [editingEntry, setEditingEntry] = useState(null);
33.    const [shareModal, setShareModal] = useState(null);
34.
35.    // Cargar entradas al montar
36.    useEffect(() => {
37.      loadEntries();
38.      // eslint-disable-next-line react-hooks/exhaustive-deps
39.    }, []);
40.
41.    /**
42.     * @function loadEntries
43.     * @description Carga todas las entradas del diario del usuario
44.     * @async
45.     */
46.    const loadEntries = async () => {
47.      setIsLoading(true);
48.      try {
49.        const data = await diarioService.getAll();
50.        setEntries(data);
51.      } catch (error) {
52.        showError(error.message || 'Error al cargar las entradas
```

```

53.         } finally {
54.             setIsLoading(false);
55.         }
56.     };
57.
58.     /**
59.      * @function handleCreateEntry
60.      * @description Maneja la creación de una nueva entrada del diario
61.      * @param {object} entryData - Los datos de la nueva entrada.
62.      * @async
63.      */
64.     const handleCreateEntry = async (entryData) => {
65.         setIsLoading(true);
66.         try {
67.             const newEntry = await diarioService.create(entryData);
68.             showSuccess('Entrada creada exitosamente!');
69.             setEntries([newEntry.entrada, ...entries]);
70.             setShowEditor(false);
71.         } catch (error) {
72.             showError(error.message || 'Error al crear la entrada');
73.         } finally {
74.             setIsLoading(false);
75.         }
76.     };
77.
78.     /**
79.      * @function handleUpdateEntry
80.      * @description Maneja la actualización de una entrada existente
81.      * @param {object} entryData - Los nuevos datos para la entrada.
82.      * @async
83.      */
84.     const handleUpdateEntry = async (entryData) => {
85.         setIsLoading(true);
86.         try {
87.             const updated = await diarioService.update(editingEntry, entryData);
88.             showSuccess('Entrada actualizada exitosamente!');
89.             setEntries(entries.map(e => e._id === editingEntry._id ? updated : e));
90.             setShowEditor(false);
91.             setEditingEntry(null);
92.         } catch (error) {
93.             showError(error.message || 'Error al actualizar la entrada');
94.         } finally {
95.             setIsLoading(false);
96.         }
97.     };
98.
99.     /**
100.      * @function handleDeleteEntry
101.      * @description Maneja la eliminación de una entrada.
102.      * @param {string} entryId - El ID de la entrada a eliminar.
103.      * @async
104.      */
105.     const handleDeleteEntry = async (entryId) => {
106.         if (!window.confirm('¿Estás seguro de que quieres eliminar esta entrada?')) {
107.             return;
108.         }
109.
110.         setIsLoading(true);
111.         try {
112.             await diarioService.delete(entryId);
113.             showSuccess('Entrada eliminada exitosamente');
114.             setEntries(entries.filter(e => e._id !== entryId));
115.         } catch (error) {
116.             showError(error.message || 'Error al eliminar la entrada');

```

```

117.         } finally {
118.             setIsLoading(false);
119.         }
120.     };
121.
122.     /**
123.      * @function handleEditEntry
124.      * @description Prepara el editor para modificar una entrada exi
125.      * @param {object} entry - La entrada a editar.
126.      */
127.     const handleEditEntry = (entry) => {
128.         setEditingEntry(entry);
129.         setShowEditor(true);
130.     };
131.
132.     /**
133.      * @function handleCancelEditor
134.      * @description Cancela la edición y cierra el editor.
135.      */
136.     const handleCancelEditor = () => {
137.         setShowEditor(false);
138.         setEditingEntry(null);
139.     };
140.
141.     /**
142.      * @function handleShareEntry
143.      * @description Abre el modal para compartir una entrada.
144.      * @param {object} entry - La entrada a compartir.
145.      */
146.     const handleShareEntry = (entry) => {
147.         setShareModal(entry);
148.     };
149.
150.     /**
151.      * @function copyShareLink
152.      * @description Copia el enlace para compartir al portapapeles.
153.      */
154.     const copyShareLink = () => {
155.         const url = `${window.location.origin}/diario/${shareModal._
156.         navigator.clipboard.writeText(url);
157.         showSuccess('¡Link copiado al portapapeles!');
158.     };
159.
160.     /**
161.      * @function closeShareModal
162.      * @description Cierra el modal de compartir.
163.      */
164.     const closeShareModal = () => {
165.         setShareModal(null);
166.     };
167.
168.     return (
169.         <div className="diario-page">
170.             <div className="diario-container">
171.                 {!showEditor ? (
172.                     <>
173.                         {/* Header */}
174.                         <div className="diario-header">
175.                             <div className="diario-header__text">
176.                                 <h1>Mi Diario Personal</h1>
177.                                 <p>Un espacio privado para tus pensa
178.                             </div>
179.                             <Button
180.                                 variant="primary"

```

```

181.         onClick={() => setShowEditor(true)}
182.         disabled={isLoading}
183.     >
184.         Nueva entrada
185.     </Button>
186. </div>
187.
188.     /* Lista de entradas */
189.     <div className="diario-content">
190.         {isLoading && entries.length === 0 ? (
191.             <div className="diario-loading">
192.                 <div className="spinner"></div>
193.                 <p>Cargando tus entradas...</p>
194.             </div>
195.         ) : entries.length === 0 ? (
196.             <div className="diario-empty">
197.                 <div className="diario-empty_ic
198.                 <h2>Tu diario está vacío</h2>
199.                 <p>Comienza a escribir tu primer
200.                 <Button
201.                     variant="primary"
202.                     onClick={() => setShowEditor
203.                 >
204.                     Crear primera entrada
205.                 </Button>
206.             </div>
207.         ) : (
208.             <div className="diario-entries">
209.                 {entries.map(entry => (
210.                     <DiaryEntry
211.                         key={entry._id}
212.                         entry={entry}
213.                         onEdit={handleEditEntry}
214.                         onDelete={handleDeleteEn
215.                         onShare={handleShareEntr
216.                     />
217.                 ))}
218.             </div>
219.         )}
220.     </div>
221. </>
222. ) : (
223.     <DiaryEditor
224.         onSave={editingEntry ? handleUpdateEntry : h
225.         onCancel={handleCancelEditor}
226.         initialData={editingEntry}
227.         isLoading={isLoading}
228.     />
229. )}
230.
231.     /* Modal de compartir */
232.     {shareModal && (
233.         <div className="modal-overlay" onClick={closeSha
234.             <div className="modal-content" onClick={(e)
235.                 <div className="modal-header">
236.                     <h2>Compartir entrada</h2>
237.                     <button className="modal-close" onCl
238.                 </div>
239.                 <div className="modal-body">
240.                     <p className="modal-description">
241.                         Comparte este link con quien qui
242.                         Necesitarán la contraseña que co
243.                     </p>
244.                     <div className="share-link-container

```

```

245.         <Input
246.             value={` ${window.location.or
247.             onChange={() => {}} // Read-
248.             icon=" "
249.         />
250.         <Button variant="primary" onClick
251.             Copiar
252.         </Button>
253.     </div>
254.     <div className="share-info">
255.         <p> <strong>Recuerda:</strong>
256.     </div>
257. </div>
258. </div>
259. </div>
260.     })
261. </div>
262. </div>
263. );
264. };
265.
266. export default Diario;

```